SEGURIDAD INFORMÁTICA · UNIVERSIDAD NACIONAL

Monitoreo de UNA Chat

OpenTelemetry, Docker y pruebas de carga Artillery

Estudiante: Gerald
Entorno: proyecto local en Docker Desktop
Fecha: 30 de octubre de 2026

Informe basado únicamente en ejecuciones y capturas comprobadas. Las integraciones externas se identifican como pendientes cuando no existe una sesión autenticada o una credencial configurada.

RESUMEN EJECUTIVO

Objetivo y estado de entrega

Instrumentar UNA Chat para producir logs estructurados, métricas y trazas; ejecutarla en contenedor; validar la mensajería Socket.IO; y someter el servicio local a carga reproducible sin afectar el despliegue público.

23	300	0
pruebas repetidas: 12 Mocha + 5 Jest + 6 Playwright (Node 24.22.1)	mensajes Socket.IO procesados	vulnerabilidades runtime

Comprobado: Docker Engine responde; el servicio Compose está healthy; interfaz HTTP responde 200; Socket.IO y el adaptador HTTP local procesaron los mensajes previstos.

Pendiente de autenticación externa: no se afirma despliegue Vercel, monitor Cronitor, evento Sentry ni recepción en Grafana Cloud. Los paneles de esas cuentas no son accesibles desde el perfil de automatización disponible en este turno.

Alcance y restricciones

- Las cargas se dirigieron exclusivamente a `127.0.0.1:3000`.
- No se detuvieron ni eliminaron contenedores ajenos.
- No se añadieron secretos al repositorio ni al informe.

PARTE 1 · ARQUITECTURA

Aplicación y protocolo

UNA Chat usa Express para servir `index.html` y Socket.IO sobre el servidor HTTP. Los clientes envían el evento `Evento-Mensaje-Server` con una cadena JSON que contiene `nombre`, `mensaje` y `color`. El servidor valida y escapa el contenido antes de emitir la respuesta.

Recorrido observado

```
Browser ── HTTP GET / ──> Express
Browser ═══ Engine.IO WebSocket ═══> Socket.IO
Client ── Evento-Mensaje-Server(JSON string) ──> validateMessage
Server ── Evento-Mensaje-Server(validated JSON) ──> connected clients
```

> **Adaptación de cliente:** el navegador usa transporte WebSocket, reconexión automática, intervalos crecientes acotados y timeout de conexión de 10 segundos.

Modificaciones principales

- `server.js`: orden de instrumentación, métricas HTTP/chat, spans y endpoint HTTP exclusivo del laboratorio local.
- `telemetry.js` y `logger.js`: SDK OpenTelemetry y Pino con IDs de traza/span.
- `index.html`: WebSocket y reconexión configurados.
- `Dockerfile`, Compose y Artillery: ejecución local reproducible.
- `vercel.json`: preparación de entrada Express y archivo estático.

PARTE 1 · SEGURIDAD

Dependencias y manejo de datos

Se auditó el grafo de dependencias con npm audit. Se corrigió el conjunto de dependencias de producción mediante la reparación compatible de lockfile y se reconstruyó la imagen.

Ámbito	Resultado comprobado	Tratamiento
Producción	0 vulnerabilidades reportadas tras npm audit --omit=dev	Imagen reconstruida con npm ci --omit=dev
Desarrollo	28 alertas: 1 alta, 24 moderadas, 3 bajas	La alta restante es serialize-javascript transitiva vía Mocha; npm solo ofrece bajar a Mocha 5.0.5, una incompatibilidad que no se aplicó

Los logs instrumentados no incorporan contenido del chat, contraseñas o tokens. Incluyen nombre de evento, transporte, resultado, ruta, duración y contexto de trazas. Los secretos OTLP quedan en variables locales no versionadas; `.env.example` contiene valores vacíos.

> **Límite:** la auditoría de producción no representa la seguridad de las dependencias de desarrollo. El archivo JSON adjunto conserva el resultado de npm audit --omit=dev; el informe de auditoría completa se conserva por separado para revisar antes de actualizar herramientas.

Vercel

Se preparó la configuración de proyecto, pero no hay evidencia de un deployment público ni de que Socket.IO haya sido probado en Vercel. El endpoint `/__lab/message` está bloqueado cuando `VERCEL=1` y no forma parte de la imagen de producción.

PARTE 2 · OPENTELEMETRY

Señales locales instrumentadas

El bootstrap `telemetry.js` se carga antes de Express y HTTP. La auto-instrumentación captura HTTP; spans manuales envuelven el procesamiento de cada mensaje Socket.IO y del adaptador HTTP de laboratorio. Pino lee el span activo y añade `trace_id` y `span_id`.

Señal	Instrumentación	Verificación local
Logs	Pino JSON + ambiente + trace/span IDs	Logs reales del contenedor archivados
Métricas	Contadores de solicitudes, errores y mensajes; histogramas de duración	Instrumentación en proceso; no exportada a nube aún
Trazas	Auto-instrumentación HTTP + spans chat.process_message	IDs correlacionados en logs; exportación remota no configurada

Grafana Cloud: el SDK permite OTLP/HTTP para los tres tipos de señal, pero no hay aún endpoint/credencial Grafana cargados. Por tanto, no se presentan capturas ni se afirma recepción remota. Para continuar se requiere completar autenticación en el navegador y configurar las variables secretas OTLP directamente en la máquina.

Privacidad y aislamiento

- Exportadores apagados si faltan endpoint y headers.
- Sin exportación desde Vercel o NODE_ENV=test.
- El contenedor Compose solo exportará a Grafana si sus variables OTLP se completan localmente.

PARTE 3 · DOCKER

Construcción y ejecución

Docker Desktop respondió con Docker Engine 29.9.2, 8 CPU y 4,024,041,472 bytes de memoria anunciada por el daemon. Compose levantó solo el servicio de UNA Chat, publicado en loopback.

```
docker info
docker compose build una-chat
docker compose up -d una-chat
docker compose ps
URL local: http://127.0.0.1:3000
```

> **Estado verificado:** contenedor `fork-ger120904-una-chat-1` healthy, sin reinicios observados; GET / devolvió HTTP 200. La app se ejecuta con Node 24.22.2 en modo producción.

Archivos para repetir

- `Dockerfile` (imagen Node 24 Alpine, dependencias de producción y healthcheck).
- `docker-compose.yml` (puerto loopback, ambiente local y variables OTLP vacías por defecto).
- `.env.example` (plantilla sin credenciales).
- `.dockerignore` (excluye módulos locales y evidencias del contexto de imagen).

El contenedor de otra aplicación se mantuvo intacto. Build final: npm ci --omit=dev reportó 0 vulnerabilidades; GET / 200 y Socket.IO validado después del rebuild.

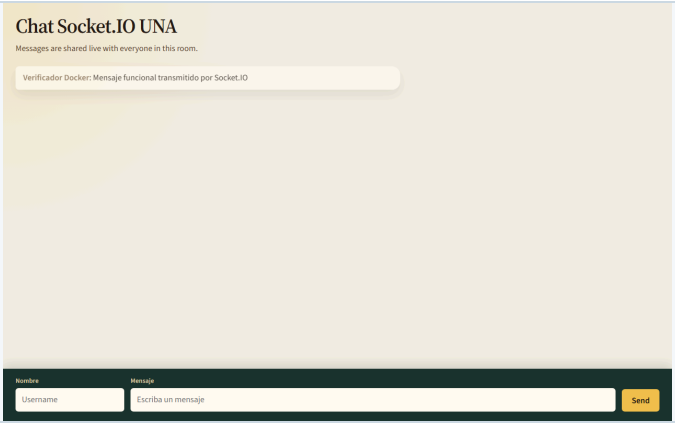
PARTE 3 · VERIFICACIÓN EN NAVEGADOR

Interfaz y conversación real

Las capturas originales se tomaron del navegador contra el servicio local de Docker. La segunda imagen muestra dos clientes reales conectados y el broadcast Socket.IO.



Evidencia 01 · portada del chat, local Docker, 2026-10-30. Captura PNG original conservada en evidencias-laboratorio/parte-3-docker-artillery/.



Evidencia 02 · mensaje emitido y recibido en dos ventanas, mismo contenedor local.

PARTE 3 · ARTILLERY SOCKET.IO

300 mensajes por el protocolo real

La prueba usó el motor integrado `socketio` de Artillery 2; no se instaló motor legado. Cada VU abre Socket.IO, envía una vez `Evento-Mensaje-Server` con JSON serializado y valida el evento de respuesta y el contenido esperado.

300
usuarios/emisiones

5/s
llegada durante 60 s

0
fallos reportados

Medición	Resultado observado
Respuestas	300 de 300
Latencia p50	7.6 ms
Latencia p95	77.5 ms
Latencia p99	111.1 ms
Procesamiento verificado en logs	300 eventos chat.message.processed

El informe bruto JSON y el YAML se adjuntan. Socket.IO no devuelve códigos HTTP por mensaje: las cifras de emisión/respuesta son eventos de aplicación, no solicitudes HTTP.

PARTE 3 · ARTILLERY HTTP

Adaptador local requerido por la guía

Como la guía solicita medir solicitudes HTTP, se creó `/__lab/message` solo para pruebas locales. La ruta reutiliza el validador real del chat y está habilitada únicamente con `LAB_HTTP_TEST_ENABLED=true`. También comprueba Host local y origen loopback/bridge. En Vercel retorna 404.

300
POST recibidos

200
respuestas exitosas

0
fallidas

Latencia	Valor
p50	7.9 ms
p95	64.7 ms
p99	671.9 ms
Tasa completada	5 solicitudes/s durante 60 s

El contador independiente encontró 300 logs con `transport=http`. El p99 alto es consistente con variabilidad de recursos, no se presenta como benchmark productivo.

> **Condición de host:** un contenedor independiente de la máquina consumió en muestras entre 49 % y 270 % de CPU, con un pico previo de 975 %. Se dejó intacto como pidió el usuario; estos resultados son reproducibles en este equipo, aunque esa interferencia puede afectar colas y percentiles de cola.

PARTE 1 · PUBLICACIÓN

Preparación de Vercel

El repositorio tiene un archivo `vercel.json` orientado al entrypoint Express y al archivo HTML. El código mantiene el servidor HTTP/Socket.IO para ofrecer WebSockets y solo llama `listen()` cuando se ejecuta como proceso local. La ruta HTTP de Artillery está excluida si `VERCEL=1`.

Estado verificado

Elemento	Estado
Configuración local y JSON de Vercel	Preparada y parseada como JSON válido
Deployment público de Vercel	**Pendiente: cuenta no vinculada al perfil operativo de navegador**
Conexión de Socket.IO pública	**No probada; no se inventa URL ni resultado**
Monitoreo Cronitor de Vercel	**Depende de obtener URL pública funcional**
Error controlado Sentry	**SDK Sentry preparado; falta proyecto/DSN autenticado y evento controlado**

Se respetará la restricción de no enviar carga a Vercel. Después de autenticar, la verificación debe ser manual/funcional de interfaz y Socket.IO con mensajes sintéticos de baja tasa.

Docs oficiales consultables: vercel.com/docs/frameworks/backend/express; vercel.com/docs/project-configuration/vercel-json; vercel.com/changelog/websocket-support-is-now-generally-available.

PARTE 2 · GRAFANA CLOUD · PARTE 1 · SENTRY · CRONITOR

Servicios externos: estado honesto

Servicio	Trabajo realizado	Falta para verificar
Grafana Cloud	SDK con exportadores OTLP de logs, métricas y trazas; Docker Compose acepta endpoint y headers por entorno.	Iniciar sesión con perfil accesible y establecer la credencial OTLP local sin compartirla por chat.
Sentry	SDK Node 10.78.0 integrado solo para Vercel; sin DSN no se inicializa. beforeSend elimina cuerpo, cookies, headers y query strings.	Crear/reutilizar proyecto y configurar DSN; generar un evento controlado desde Vercel y apagar la ruta temporal protegida.
Cronitor	Cuenta abierta en navegador compartido previamente.	Crear monitor después de que exista URL Vercel pública y verificar la notificación.

Sin telemetría remota afirmada: los registros locales y la instrumentación están comprobados. No hay captura real de un panel Grafana con esta telemetría en este estado. El informe no contiene claves, DSN, tokens ni capturas de sesión que puedan revelarlos.

Autenticación Grafana requerida mediante la página segura de acceso a perfil que se comparte con el usuario en el mensaje final. Tras iniciar sesión debe pulsarse **Save profile** y avisar en este chat.

REPRODUCCIÓN Y CONCLUSIÓN

Entrega técnica y siguientes verificaciones

Comandos reproducibles

```
docker compose build una-chat
docker compose up -d una-chat
npm run test:load:socketio
npm run test:load:http
docker compose logs --no-color una-chat
```

Evidencias

- PNG originales de UI y dos ventanas bajo `evidencias-laboratorio/parte-3-docker-artillery/`.
- JSON de resultados Artillery Socket.IO y HTTP; YAMLs de prueba.
- Logs Pino del contenedor, verificación Docker y auditoría de dependencias de producción.

Conclusión

La parte local del laboratorio queda implementada y medida: la imagen Docker está saludable y el protocolo real produjo las 300 respuestas esperadas; el adaptador HTTP local produjo 300/300 respuestas correctas. La operación completa en nube sigue abierta. Vercel, Cronitor, Sentry y Grafana Cloud no se declaran terminados sin publicar/autenticar, configurar credenciales y comprobar paneles/eventos reales.

> **Limitación documental:** no se incluyen resultados externos no observados ni se afirma cumplir aquellas partes del laboratorio que dependen de autenticación o URL pública.